



Progetto PWM “Fast Food” - A.A. 2025/2026

Autore: Rimoldi Diego

Introduzione del Progetto:

Per avviare l'applicazione, dalla root del progetto installa le dipendenze con:

`npm install`

Poi avvia il backend Express in modalità sviluppo:

`npm run dev`

In alternativa, per un avvio standard senza nodemon:

`npm start`

Una volta avviato il server, apri nel browser **`http://localhost:3000`** per raggiungere la pagina iniziale.

La documentazione Swagger è disponibile su **`http://localhost:3000/swagger`**.

Struttura della Directory:

```
├── documents
│   ├── meals.json
│   ├── PWM_project_25_26.pdf
│   ├── relazione_progettuale.md
│   ├── relazione_tecnica.md
│   ├── swagger.js
│   └── swagger.json
├── middlewares
│   ├── authenticateUser.js
│   └── authorizeRistoratore.js
├── node_modules
├── public
│   ├── assets
│   │   ├── auth.js
│   │   ├── Favicon.png
│   │   ├── Logo.png
│   │   ├── modern-theme.css
│   │   └── responsive.css
```

- └─ cliente
 - ├─ carrello.html
 - ├─ home.html
 - ├─ menù.html
 - ├─ offerte.html
 - └─ ordini.html
- └─ ristoratore
 - ├─ creaRistorante.html
 - ├─ gestioneBacheca.html
 - ├─ gestioneMenù.html
 - ├─ gestioneRistorante.html
 - ├─ home.html
 - ├─ modificaPiattoPersonalizzato.html
 - ├─ ordini.html
 - ├─ piattiGenerici.html
 - ├─ piattoPersonalizzato.html
 - └─ statistiche.html
- ├─ index.html
- ├─ login.html
- ├─ logout.html
- ├─ profilo.html
- └─ register.html
- └─ README
 - ├─ README.pdf
 - ├─ relazione_progettuale.pdf
 - ├─ relazione_tecnica.pdf
 - └─ Video_Dimostrativo_Progettuale.mp4
- └─ routes
 - ├─ carts.js
 - ├─ meals.js
 - ├─ orders.js
 - ├─ restaurants.js
 - └─ users.js
- └─ utils
 - ├─ addressValidation.js
 - └─ config.js
- ├─ .env
- ├─ .gitattributes
- ├─ .gitignore
- ├─ index.js
- ├─ package-lock.json
- ├─ package.json
- └─ README.md

Struttura del Database:

Il progetto utilizza **MongoDB Atlas** con database *Fast-Food*, accessibile dal backend Express tramite le variabili presenti nel file **.env**.

Collections:

Nel database sono presenti 5 collezioni principali:

- **users:** anagrafica utenti registrati (clienti e ristoratori), con ruolo (role) e password salvata in hash.
- **restaurants:** dati dei ristoranti creati dai ristoratori, con informazioni di profilo e riferimenti ai piatti gestiti.
- **meals:** catalogo piatti (generici + personalizzati); i piatti personalizzati sono legati al ristorante tramite ristorante_id.
- **carts:** carrelli persistenti associati all'utente, con quantità e raggruppamento per ristorante.
- **orders:** ordini effettuati dagli utenti, con stato di avanzamento (es. ordinato, in preparazione, in consegna, consegnato).

Questa struttura è coerente con i router API presenti in routes/ (**users.js**, **restaurants.js**, **meals.js**, **carts.js**, **orders.js**).

Memorizzazione "Carrello" nel DB:

Ho scelto di salvare il carrello nella collection carts del database invece che nel localStorage.

In questo modo il carrello resta associato all'account utente anche dopo logout o cambio dispositivo, garantendo un comportamento più simile a una piattaforma e-commerce reale.

Middlewares:

Per proteggere le API ho separato i controlli in due middleware dedicati:

- **authenticateUser.js:** verifica presenza e validità del token JWT nell'header Authorization.
- **authorizeRistoratore.js:** consente l'accesso solo agli utenti con ruolo ristoratore nelle route riservate.

Questa divisione mi permette di riutilizzare la stessa autenticazione su più endpoint e applicare autorizzazioni diverse in base al contesto.

Swagger:

La documentazione API Swagger è integrata nel progetto ed è disponibile dopo l'avvio del server:

- URL: **http://localhost:3000/swagger**
- Configurazione/generazione: **documents/swagger.js** e **documents/swagger.json**
- Avvio applicazione (dev): **npm run dev**

In questo modo è possibile testare rapidamente gli endpoint REST del progetto direttamente da interfaccia web.

Scelte Implementative:

Questa sezione descrive le scelte architetturali adottate nel progetto **Fast Food**, in base a come ho strutturato backend, autenticazione e dati applicativi.

Utilizzo di token JWT:

Per gestire autenticazione e autorizzazioni ho scelto **JWT**: dopo il login su **POST /users/login**, il server valida le credenziali e genera un token firmato con chiave segreta (**JWT_SECRET**) e scadenza (**JWT_EXPIRES_IN**) definite nel file **.env**.

Nel payload del token vengono salvati **userId** e **role**, così da poter distinguere in modo immediato cliente e ristoratore durante l'accesso alle API protette.

Il token viene restituito al client, memorizzato lato Frontend e inviato nelle richieste successive tramite **header Authorization: Bearer <token>**.

Hashing delle Password:

Le password non vengono mai salvate in chiaro: in fase di registrazione utilizzo **bcrypt** per creare l'hash, mentre in fase di login confronto password inserita e hash memorizzato.

Questa scelta aumenta la sicurezza dei dati utente anche in caso di accesso non autorizzato al database.

Relazioni con entità Ristorante e Ristoratore:

Nel modello dati ho impostato una relazione 1:1 tra **ristoratore** e **ristorante**: ogni account ristoratore può creare e gestire un solo ristorante.

I piatti sono centralizzati nella collection **meals**:

- I piatti **generici** hanno **ristorante_id = null**;
- I piatti **personalizzati** hanno **ristorante_id** valorizzato con l'id del ristorante proprietario.

In questo modo posso riutilizzare un'unica collection per il catalogo completo, mantenendo però la distinzione tra contenuti globali e contenuti gestiti dal singolo ristoratore.

Gestione Piatti Personalizzati:

I piatti personalizzati non sono in una collection separata: uso la stessa collection **meals** dei piatti generici.

La distinzione avviene tramite il campo **ristorante_id**:

- null per i piatti generici;
- Valorizzato con ID ristorante per i piatti creati da un ristoratore.

Questa scelta semplifica query, filtri e manutenzione dei dati.

Sistema degli Ordini:

Nel carrello i prodotti vengono raggruppati per ristorante. In fase di conferma ordine, il sistema genera gli ordini in modo coerente con il ristorante selezionato e con la modalità scelta dall'utente:

- **ritiro in ristorante;**
- **consegna a domicilio.**

Questo permette di gestire più ristoranti nello stesso flusso senza perdere la separazione operativa degli ordini.

Ritiro in Ristorante:

Per gli ordini con ritiro, il flusso degli stati è:

ordinato → in preparazione → consegnato

L'avanzamento è gestito dal ristorante tramite la propria area di gestione ordini.

Consegna a Domicilio:

Per la consegna a domicilio, il flusso è:

ordinato → in preparazione → in consegna → consegnato

Il ristorante porta l'ordine fino a in consegna; la conferma finale di consegnato viene effettuata dal cliente alla ricezione.

Per entrambe le modalità, quando un ordine è il primo in coda per un ristorante, passa automaticamente allo stato in preparazione.

Bacheca delle Offerte:

Il progetto include un flusso completo per la gestione promozioni:

- **Ristoratore:** crea e pubblica offerte dalla pagina **public/ristoratore/gestioneBacheca.html**.
- **Cliente:** visualizza le promozioni attive nella pagina **public/cliente/offerte.html**.
- **Backend:** salva e recupera i dati delle offerte tramite API REST e con il collegamento al database di MongoDB.

Offerte lato Cliente (public/cliente/offerte.html):

La pagina offerte cliente:

- Raggruppa i piatti in promozione per categoria;
- Mostra prezzo originale, prezzo scontato e risparmio;
- Consente inserimento diretto nel carrello al prezzo promozionale.

Dimostrazione:

```
<title>Fast Food - Offerte in Bacheca</title>
```

```
...
```

```
<div class="offer-price-row">  
  <span class="offer-old-price">€ ...</span>  
  <span class="offer-arrow">></span>  
  <span class="offer-new-price">€ ...</span>  
  <span class="offer-save">Risparmi € ...</span>  
</div>
```

```
async function addToCart(meal) {  
  var payload = {  
    meal_id: meal._id,  
    nome: meal.strMeal,  
    quantita: quantita,  
    prezzo_unitario: meal.prezzo_scontato,
```

```

    prezzo_originale: meal.prezzo,
    in_offerta: true,
    sconto_percentuale: meal.sconto_percentuale
  };
  var response = await fetchWithAuth(apiBase + "/carts/me/items", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(payload)
  });
}

```

Offerte lato Ristoratore (public/ristoratore/gestioneBacheca.html):

La pagina bacheca ristoratore è pensata per:

- Selezionare piatti del menù;
- Applicare percentuali di sconto;
- Evidenziare chiaramente i piatti già in offerta.

Dimostrazione:

```

.menu-card.in-offerta {
  background: linear-gradient(160deg, #ffffff 0%, #f2fff8 100%);
  outline-color: rgba(25, 135, 84, 0.4);
}

.offer-badge {
  background: linear-gradient(120deg, #198754 0%, #20c997 100%);
  color: #fff;
  font-weight: 700;
}

```

Funzioni Frontend e Backend del Progetto:

HTML5:

Frontend (public/cliente/offerte.html):

```

<!DOCTYPE html>
<html lang="it">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />

```

CSS3:

Frontend (public/assets/modern-theme.css):

```

:root {
  --brand-primary: #0d6efd;
  --brand-accent: #fec44c;
}

```

```
.sidebar a.nav-link.active {
  background: linear-gradient(120deg, var(--brand-primary) 0%, #6ea8fe 100%) !important;
  color: #fff !important;
}
```

JSON:

```
const swaggerDocument = JSON.parse(fs.readFileSync("./documents/swagger.json", "utf-8"));
...
const rawMeals = JSON.parse(fs.readFileSync("./documents/meals.json", "utf-8"));
```

JavaScript (JS):

Frontend (public/cliente/offerte.html):

```
grouped[category].forEach(function(meal) {
  var col = document.createElement("div");
  ...
  col.querySelector("button").addEventListener("click", function() {
    addToCart(meal);
  });
});
```

Backend (routes/users.js):

```
const token = jwt.sign(
  { userId: user._id.toString(), role: user.role },
  process.env.JWT_SECRET,
  { expiresIn: process.env.JWT_EXPIRES_IN }
);
```

Swagger API:

Dimostrazione nel Progetto di (documents/swagger.js):

```
const outputFile = './documents/swagger.json';
const inputFiles = ['./index.js', './routes/users.js', './routes/meals.js', './routes/restaurants.js', './routes/orders.js', './routes/carts.js'];
swaggerAutogen(outputFile, inputFiles, doc);
```

API OpenStreetMap:

Backend (utils/addressValidation.js):

```
const NOMINATIM_BASE_URL = "https://nominatim.openstreetmap.org/search";
...
const response = await fetch(url, {
  headers: {
    "User-Agent": USER_AGENT,
    "Accept": "application/json"
  },
  signal: controller.signal
});
```

NodeJS:

Backend (index.js):

```
const app = express();
app.use(express.json());
app.use(cors());
app.use(express.static(path.join(__dirname, "public")));
```

MongoDB:

Backend (index.js + routes/restaurants.js):

```
const client = new MongoClient(config.MONGODB_URI);
await client.connect();
app.locals.db = client.db(config.MONGODB_DB);

const restaurants = await db.collection("restaurants")
  .find(filter)
  .toArray();
```

Paradigma REST:

Backend (routes/restaurants.js e routes/users.js):

```
router.get("/search", async (req, res) => { ... });
usersRouter.post("/register", async (req, res) => { ... });
usersRouter.post("/login", async (req, res) => { ... });
usersRouter.get("/me", authenticateUser, async (req, res) => { ... });
usersRouter.put("/me", authenticateUser, async (req, res) => { ... });
```